Unidad I: Árboles

Conceptos, recorridos, BST, balanceo y STL

Ing. Irving Cardona Camacho

UAA

11 de agosto de 2026

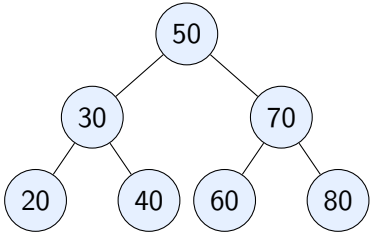
Estructuras lineales vs. no lineales

Estructura lineal



- Lista
- Pila
- Cola

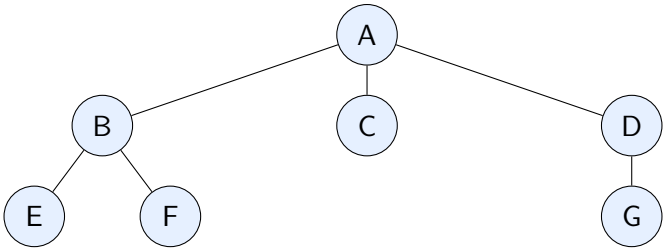
Estructura no lineal



- Árboles
- Grafos

¿Qué es un árbol?

- Un **árbol** es una estructura de datos jerárquica.
- Está formado por **nodos**.
- Existe un nodo principal llamado **raíz**.
- Cada nodo puede tener hijos.
- Un árbol no debe tener ciclos.



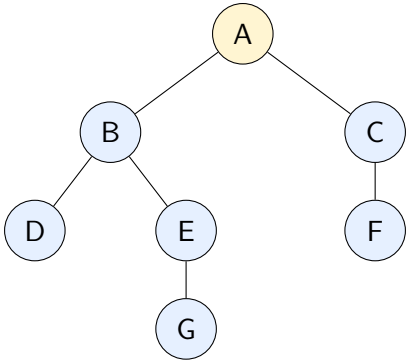
¿Dónde se usan los árboles?

- Sistemas de archivos.
- Menús jerárquicos.
- Árboles genealógicos.
- Expresiones matemáticas.
- Compiladores.
- Bases de datos.
- Índices de búsqueda.
- Contenedores ordenados de C++.

Términos básicos

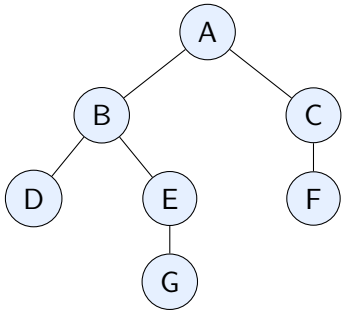
- **Raíz**: nodo principal del árbol.
- **Padre**: nodo que tiene uno o más hijos.
- **Hijo**: nodo que depende directamente de otro.
- **Hermanos**: nodos que comparten el mismo padre.
- **Hoja**: nodo sin hijos.
- **Nodo interno**: nodo que tiene al menos un hijo.
- **Subárbol**: árbol formado a partir de un nodo.

Ejemplo de terminología



- Raíz: A.
- Hijos de A: B y C.
- Hojas: D, G y F.
- Nodos internos: A, B, C y E.

Nivel, profundidad y altura



- **Nivel**: capa donde está el nodo.
- **Profundidad**: distancia desde la raíz.
- **Altura**: camino más largo hacia una hoja.

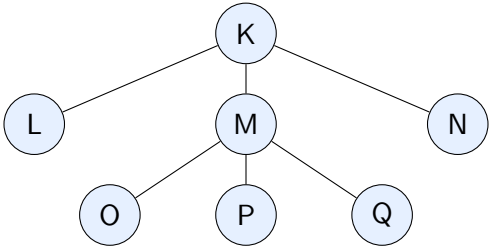
$$profundidad(A) = 0$$

$$profundidad(G) = 3$$

$$altura(A) = 3$$

Grado de un nodo

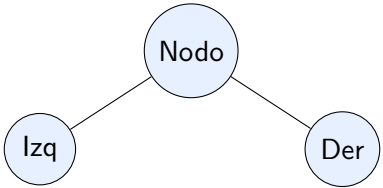
- El **grado** de un nodo es la cantidad de hijos que tiene.



- Grado de K: 3.
- Grado de M: 3.
- Grado de L: 0.
- Grado máximo del árbol: 3.

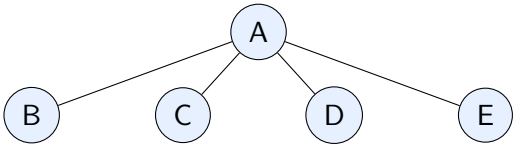
Árbol binario

- Un **árbol binario** es un árbol donde cada nodo puede tener como máximo dos hijos.
- Los hijos suelen llamarse:
  - Hijo izquierdo.
  - Hijo derecho.



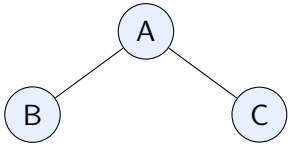
Árbol general vs. árbol binario

Árbol general



Un nodo puede tener varios hijos.

Árbol binario

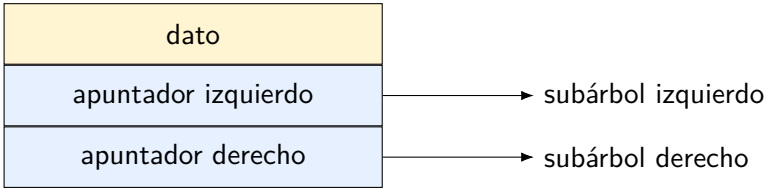


Un nodo puede tener máximo dos hijos.

Tipos comunes de árboles binarios

- **Lleno**: cada nodo tiene 0 o 2 hijos.
- **Perfecto**: todos los niveles están completamente llenos.
- **Completo**: todos los niveles están llenos excepto quizá el último, que se llena de izquierda a derecha.
- **Balanceado**: la altura se mantiene controlada.

Representación conceptual de un nodo



- Cada nodo almacena un dato.
- También guarda referencias hacia otros nodos.

Representación en C++

```cpp
struct Nodo {
    int dato;
    Nodo* izquierdo;
    Nodo* derecho;
};
```

- `dato`: información almacenada.
- `izquierdo`: dirección del hijo izquierdo.
- `derecho`: dirección del hijo derecho.
- `nullptr`: indica ausencia de hijo.

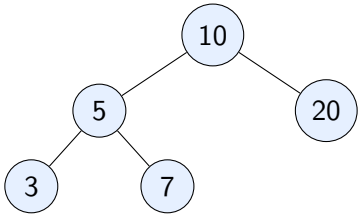
Creación de un nodo

```
1 Nodo* crearNodo(int dato) {
2     Nodo* nuevo = new Nodo;
3
4     nuevo->dato = dato;
5     nuevo->izquierdo = nullptr;
6     nuevo->derecho = nullptr;
7
8     return nuevo;
9 }
```

- Se reserva memoria dinámica con new.
- El nodo inicialmente no tiene hijos.

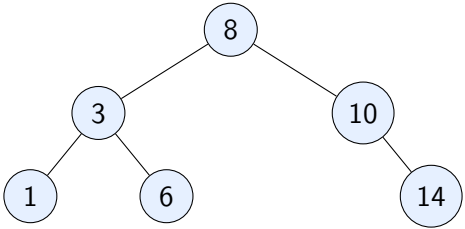
Construcción manual de un árbol

```
1 Nodo* raiz = crearNodo(10);
2
3 raiz->izquierdo = crearNodo(5);
4 raiz->derecho = crearNodo(20);
5
6 raiz->izquierdo->izquierdo = crearNodo(3);
7 raiz->izquierdo->derecho = crearNodo(7);
```



¿Por qué recorrer un árbol?

- En una lista se avanza de nodo en nodo.
- En un árbol existen varias rutas posibles.
- Recorrer un árbol significa visitar todos sus nodos en cierto orden.

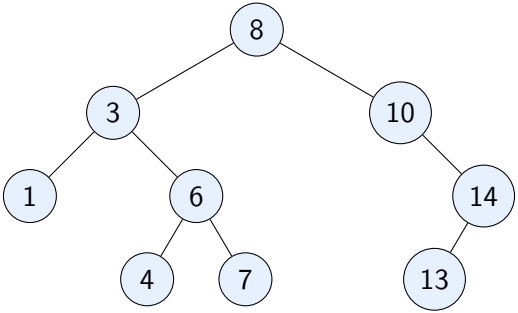


DFS: recorrido en profundidad

- DFS significa *Depth-First Search*.
- Primero se avanza hacia lo más profundo del árbol.
- En árboles binarios se estudian tres recorridos clásicos:

Recorrido	Orden
Preorden	Raíz, Izquierda, Derecha
Inorden	Izquierda, Raíz, Derecha
Postorden	Izquierda, Derecha, Raíz

Árbol base para los recorridos



Preorden

Raíz → Izquierda → Derecha

```
1 void preorden(Nodo* nodo) {
2     if (nodo == nullptr)
3         return;
4
5     cout << nodo->dato << " ";
6     preorden(nodo->izquierdo);
7     preorden(nodo->derecho);
8 }
```

8, 3, 1, 6, 4, 7, 10, 14, 13

Inorden

Izquierda $\rightarrow$ Raíz $\rightarrow$ Derecha

```
1 void inorden(Nodo* nodo) {
2     if (nodo == nullptr)
3         return;
4
5     inorden(nodo->izquierdo);
6     cout << nodo->dato << " ";
7     inorden(nodo->derecho);
8 }
```

1, 3, 4, 6, 7, 8, 10, 13, 14

Postorden

Izquierda → Derecha → Raíz

```
1 void postorden(Nodo* nodo) {
2     if (nodo == nullptr)
3         return;
4
5     postorden(nodo->izquierdo);
6     postorden(nodo->derecho);
7     cout << nodo->dato << " ";
8 }
```

1, 4, 7, 6, 3, 13, 14, 10, 8

Resumen visual de recorridos DFS

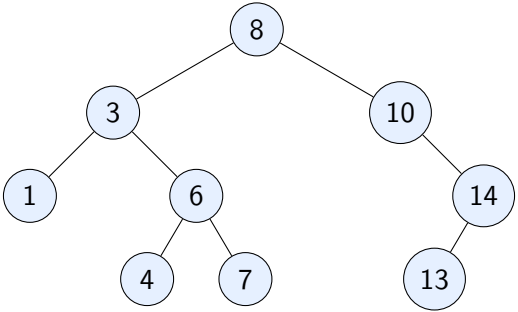
Recorrido	Regla	Uso común
Preorden	Raíz primero	Copiar árboles
Inorden	Raíz en medio	BST ordenado
Postorden	Raíz al final	Liberar memoria

Idea clave

Los tres recorridos usan recursividad, pero cambian el momento en que se procesa la raíz.

BFS: recorrido por niveles

- BFS significa *Breadth-First Search*.
- Visita el árbol por niveles.
- Usa una cola.



8, 3, 10, 1, 6, 14, 4, 7, 13

Idea del algoritmo BFS

1. Insertar la raíz en una cola.
2. Mientras la cola no esté vacía:
   1. Sacar el primer nodo.
   2. Procesarlo.
   3. Insertar su hijo izquierdo si existe.
   4. Insertar su hijo derecho si existe.

Relación con Estructuras de Datos I

BFS reutiliza el concepto de cola: primero en entrar, primero en salir.

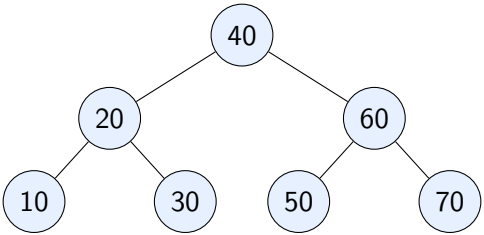
BFS usando std::queue

```
1  #include <queue>
2
3  void bfs(Nodo* raiz) {
4      if (raiz == nullptr)
5          return;
6
7      queue<Nodo*> cola;
8      cola.push(raiz);
9
10     while (!cola.empty()) {
11         Nodo* actual = cola.front();
12         cola.pop();
13
14         cout << actual->dato << " ";
15
16         if (actual->izquierdo != nullptr)
17             cola.push(actual->izquierdo);
```

Árbol Binario de Búsqueda

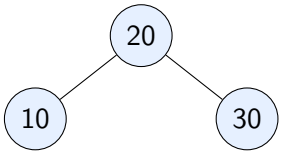
- Un **Árbol Binario de Búsqueda** también se conoce como **BST**.
- Es un árbol binario con una propiedad especial:

$$\text{Valores izquierda} < \text{Nodo} < \text{Valores derecha}$$



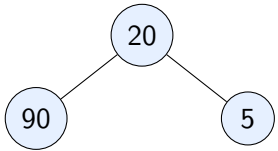
No todo árbol binario es BST

Sí es BST



$10 < 20 < 30$

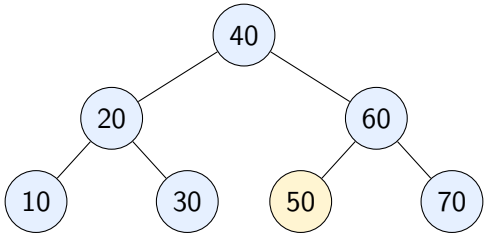
No es BST



$90 \not< 20, \quad 520$

Búsqueda en un BST

- Si el valor buscado es igual al nodo actual, se encontró.
- Si es menor, se busca por la izquierda.
- Si es mayor, se busca por la derecha.



Buscar 50:

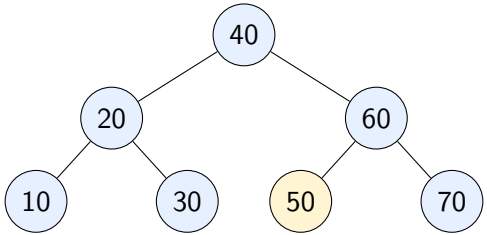
$$40 \rightarrow 60 \rightarrow 50$$

Búsqueda recursiva en BST

```
1  bool buscar(Nodo* raiz, int valor) {
2      if (raiz == nullptr)
3          return false;
4
5      if (raiz->dato == valor)
6          return true;
7
8      if (valor < raiz->dato)
9          return buscar(raiz->izquierdo, valor);
10     else
11         return buscar(raiz->derecho, valor);
12 }
```

Inserción en un BST

- La inserción sigue la misma lógica que la búsqueda.
- Si el valor es menor, se avanza a la izquierda.
- Si el valor es mayor, se avanza a la derecha.
- Cuando se encuentra un espacio vacío, se inserta el nuevo nodo.



Insertar 50:

$$40 \rightarrow 60 \rightarrow \textit{izquierda}$$

Inserción en BST

```
1  Nodo* insertar(Nodo* raiz, int valor) {
2      if (raiz == nullptr)
3          return crearNodo(valor);
4
5      if (valor < raiz->dato)
6          raiz->izquierdo = insertar(raiz->izquierdo, valor);
7      else if (valor > raiz->dato)
8          raiz->derecho = insertar(raiz->derecho, valor);
9
10     return raiz;
11 }
```

Nota

En este ejemplo no se insertan valores repetidos.

Eliminación en un BST

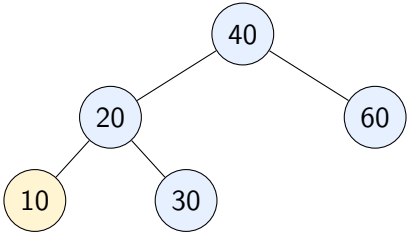
Eliminar un nodo puede tener tres casos:

1. El nodo es hoja.
2. El nodo tiene un solo hijo.
3. El nodo tiene dos hijos.

Caso más importante

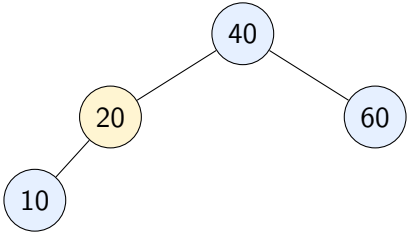
Cuando el nodo tiene dos hijos, suele reemplazarse por su sucesor inorden.

Eliminación: nodo hoja



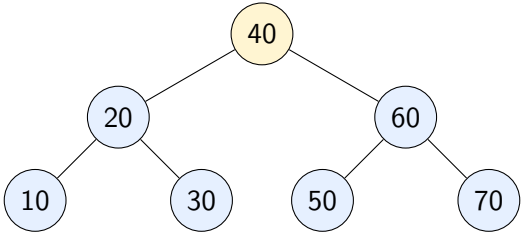
- El nodo 10 no tiene hijos.
- Se elimina directamente.

Eliminación: nodo con un hijo



- El nodo 20 tiene un solo hijo.
- El hijo ocupa su lugar.

Eliminación: nodo con dos hijos



- El nodo 40 tiene dos hijos.
- Se puede reemplazar por el menor valor del subárbol derecho.
- En este caso, el sucesor inorden es 50.

Complejidad en un BST

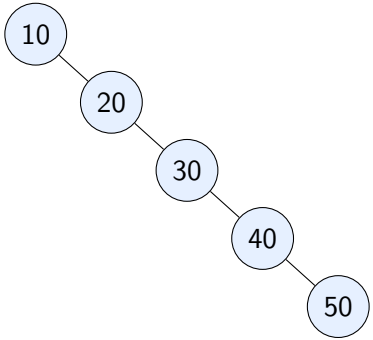
Operación	Árbol balanceado	Árbol degradado
Buscar	$O(\log n)$	$O(n)$
Insertar	$O(\log n)$	$O(n)$
Eliminar	$O(\log n)$	$O(n)$

Idea clave

El rendimiento depende directamente de la altura del árbol.

Degradación del BST a lista enlazada

Insertar valores ordenados puede producir un árbol muy desbalanceado.



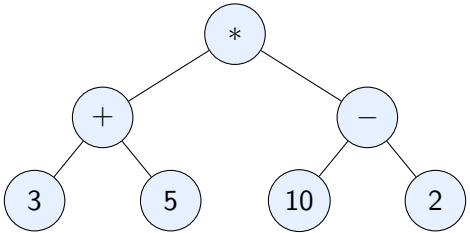
10, 20, 30, 40, 50

- La búsqueda deja de parecerse a búsqueda binaria.
- Se comporta como una lista.

Árboles de expresión

- Representan expresiones matemáticas.
- Los operadores suelen estar en nodos internos.
- Los operandos suelen estar en hojas.

$$(3 + 5) * (10 - 2)$$



Notaciones en árboles de expresión

Para la expresión:

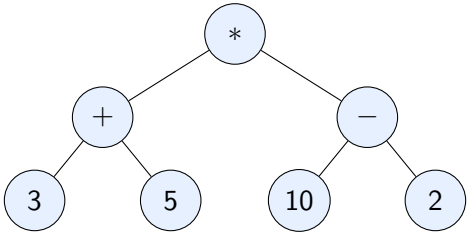
$$(3 + 5) * (10 - 2)$$

Notación	Resultado
Infija	$(3 + 5) * (10 - 2)$
Prefija	$* + \ 3\ 5 \ - \ 10\ 2$
Postfija	$3\ 5 \ + \ 10\ 2 \ - \ *$

Relación con recorridos

- Preorden produce notación prefija.
- Inorden produce notación infija.
- Postorden produce notación postfija.

Evaluación de un árbol de expresión



$$3 + 5 = 8$$

$$10 - 2 = 8$$

$$8 * 8 = 64$$

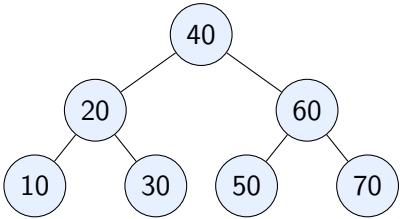
Idea clave

La evaluación natural de un árbol de expresión se parece al recorrido postorden

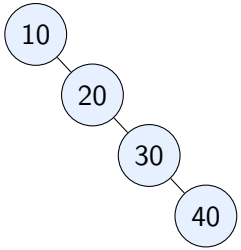
¿Por qué balancear un árbol?

- Un BST eficiente tiene altura pequeña.
- Si el árbol se desbalancea, las operaciones empeoran.
- Los árboles balanceados intentan mantener altura cercana a $\log n$.

Balanceado



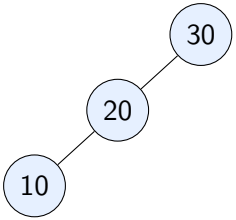
Desbalanceado



Rotaciones

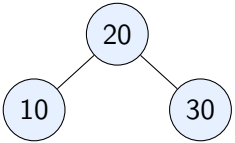
- Una rotación reorganiza nodos sin romper la propiedad del BST.
- Sirve para reducir desbalance.
- Hay rotaciones simples y dobles.

Antes



$\Rightarrow$

Después



Árbol AVL

- Un árbol AVL es un BST auto-balanceado.
- Controla la diferencia de altura entre subárboles.
- Su factor de balance suele definirse como:

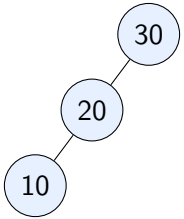
$$FB = altura(izquierdo) - altura(derecho)$$

Condición AVL

Para cada nodo, el factor de balance debe estar entre -1 y 1.

$$FB \in \{-1, 0, 1\}$$

Ejemplo de desbalance AVL

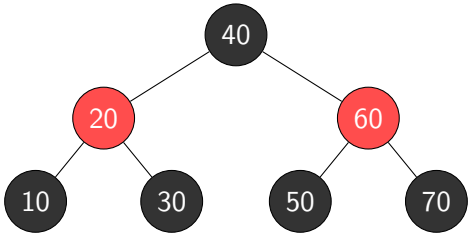


- El árbol se cargó hacia la izquierda.
- Una rotación derecha puede equilibrarlo.

$$30 \leftarrow 20 \leftarrow 10$$

Árbol Rojo-Negro

- Es otro tipo de árbol BST auto-balanceado.
- Cada nodo tiene un color: rojo o negro.
- Usa reglas de color para mantener el árbol balanceado.
- Es común en implementaciones de contenedores ordenados.



AVL vs. Rojo-Negro

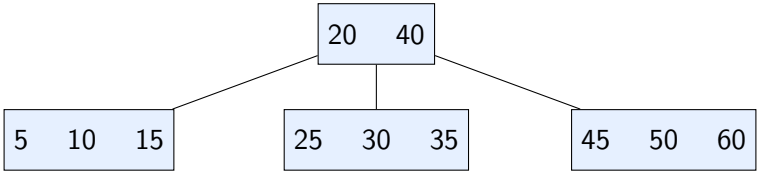
Aspecto	AVL	Rojo-Negro
Balance	Más estricto	Menos estricto
Búsqueda	Muy eficiente	Eficiente
Inserción/eliminación	Más rotaciones	Menos rotaciones
Uso común	Teórico y práctico	STL ordenada

Idea clave

Ambos buscan evitar que el BST se degrade a una lista.

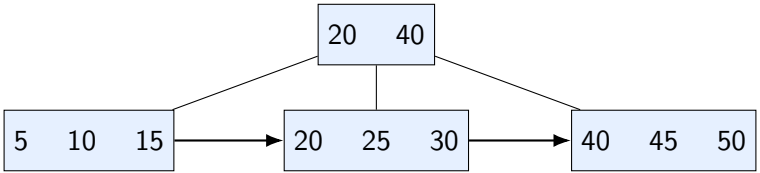
Árboles B

- Son árboles balanceados diseñados para almacenamiento secundario.
- Cada nodo puede guardar varias claves.
- Reducen la cantidad de accesos a disco.
- Son muy usados en bases de datos y sistemas de archivos.



Árboles B+

- Variante de los árboles B.
- Los datos suelen almacenarse en las hojas.
- Las hojas se conectan entre sí para recorridos secuenciales.
- Son comunes en índices de bases de datos.



Árbol B vs. Árbol B+

Aspecto	Árbol B	Árbol B+
Datos	En nodos internos y hojas	Principalmente en hojas
Hojas enlazadas	No necesariamente	Sí
Recorrido secuencial	Menos directo	Más eficiente
Uso	Índices generales	Bases de datos

Contenedores asociativos ordenados

- La STL ofrece estructuras ya implementadas.
- Algunas mantienen sus elementos ordenados.
- Internamente suelen estar basadas en árboles balanceados.

Contenedor	Uso
`std::set`	Conjunto sin repetidos
`std::multiset`	Conjunto con repetidos
`std::map`	Diccionario clave-valor
`std::multimap`	Diccionario con claves repetidas

std::set

```
1 #include <set>
2
3 set<int> numeros;
4
5 numeros.insert(40);
6 numeros.insert(10);
7 numeros.insert(30);
8 numeros.insert(10);
```

- No permite elementos repetidos.
- Mantiene los datos ordenados.

10, 30, 40

std::multiset

```
1 #include <set>
2
3 multiset<int> numeros;
4
5 numeros.insert(40);
6 numeros.insert(10);
7 numeros.insert(30);
8 numeros.insert(10);
```

- Permite elementos repetidos.
- Mantiene los datos ordenados.

10, 10, 30, 40

std::map

```cpp
#include <map>
#include <string>

map<string, int> edades;

edades["Ana"] = 20;
edades["Luis"] = 22;
edades["Carlos"] = 19;
```

- Asocia una clave con un valor.
- Las claves no se repiten.
- Las claves se mantienen ordenadas.

std::multimap

```cpp
#include <map>
#include <string>

multimap<string, string> cursos;

cursos.insert({"ISC", "Estructuras"});
cursos.insert({"ISC", "Programacion"});
cursos.insert({"IM", "Calculo"});
```

- Permite claves repetidas.
- Útil cuando una clave puede tener varios valores.

Relación entre `map` y `set`

- `set` almacena valores únicos.
- `map` almacena pares clave-valor.
- Ambos mantienen orden.
- Ambos ofrecen búsqueda eficiente.

Contenedor	Modelo mental
`set<int>`	Árbol de enteros
`map<string,int>`	Árbol ordenado por clave

Operaciones frecuentes

```
auto it = numeros.find(30);

if (it != numeros.end()) {
    cout << "Encontrado";
}

numeros.erase(10);
```

Operación	Significado
`insert()`	Insertar
`find()`	Buscar
`erase()`	Eliminar
`end()`	Posición posterior al último elemento

Nota: `map` vs. `unordered_map`

Contenedor	Orden	Estructura mental
`map`	Sí	Árbol balanceado
`unordered_map`	No	Tabla hash

Importante

En esta unidad interesa principalmente `map`, porque se relaciona con árboles balanceados.